

# SPRINT 13 BACKLOG

Investment Manager - Rentabilidade ponderada da carteira

## Sprint Goal

Calcular e apresentar a rentabilidade ponderada da carteira, neutralizando distorções causadas por compras e vendas e considerando os proventos recebidos.

## 1. Objetivo da Sprint

Evoluir o Investment Manager para calcular a rentabilidade histórica da carteira por metodologia ponderada, reutilizando a infraestrutura de posições e cotações históricas criada na Sprint 12.

A rentabilidade deverá neutralizar o efeito das movimentações de compra e venda e considerar os proventos como retorno produzido pelos investimentos.

## 2. Escopo funcional

- Rentabilidade total da carteira.
- Rentabilidade dos últimos 12 meses.
- Rentabilidade do último mês.
- Série histórica percentual da rentabilidade.
- Gráfico de evolução da rentabilidade.
- Tabela de rentabilidade mensal, retorno anual e acumulado.
- Composição geométrica dos retornos diários.

## 3. Regras de negócio

- O Investment Manager permanece um gerenciador/consolidador de investimentos, e não uma conta de corretora.
- Não criar controle de caixa, saldo disponível, DEPOSIT, WITHDRAWAL ou PortfolioCashFlow.
- BUY não depende de saldo previamente cadastrado e não deve gerar rentabilidade artificial.
- SELL não deve gerar perda artificial apenas pela redução do patrimônio investido.
- INCOME representa retorno do investimento e deve ser considerado na rentabilidade.
- Custos operacionais, corretagem e taxas não entram nesta Sprint.
- Retornos de vários dias devem ser compostos geometricamente, e não somados.

## 4. Formula planejada

Para cada dia  $d$ , a regra-base planejada é:

$$R_d = (V_d + S_d + D_d) / (V_{d-1} + B_d) - 1$$

- $V_{d-1}$ : valor de mercado da carteira no fechamento do dia anterior.
- $V_d$ : valor de mercado da carteira no fechamento do dia atual, após as operações.
- $B_d$ : valor das compras realizadas no dia.
- $S_d$ : valor das vendas realizadas no dia.
- $D_d$ : proventos considerados no dia.

Rentabilidade acumulada:

$$TWR = (1 + R_1) \times (1 + R_2) \times \dots \times (1 + R_n) - 1$$

## 5. Contrato planejado da API

Endpoint específico de rentabilidade para a carteira autenticada, com desenho final definido durante a implementação.

- Resposta com rentabilidade total, últimos 12 meses e último mês.
- Série histórica percentual para o gráfico.
- Dados mensais suficientes para retorno mensal, anual e acumulado.
- Ownership e tratamento de erros seguindo o padrão atual da API.

## 6. User Stories

- US-13.1: visualizar a rentabilidade total da carteira.
- US-13.2: visualizar a rentabilidade dos últimos 12 meses e do último mês.
- US-13.3: acompanhar a evolução histórica percentual em gráfico.
- US-13.4: consultar rentabilidade mensal, retorno anual e acumulado.
- US-13.5: ter compras e vendas neutralizadas no cálculo de performance.
- US-13.6: ter proventos considerados como retorno do investimento.

## 7. Critérios de aceite

- Compra isolada sem variação econômica não gera ganho artificial.
- Venda isolada sem variação econômica não gera perda artificial.
- Provento aumenta corretamente a rentabilidade.
- Retornos diários são compostos geometricamente.
- Série respeita usuário autenticado e carteira selecionada.
- Cards, gráfico e tabela apresentam valores consistentes com o backend.
- Funcionalidades existentes permanecem funcionando.

## 8. Tarefas tecnicas previstas

### Backend

- Criar modelos/DTOs da rentabilidade historica.
- Implementar servico de calculo diario da rentabilidade ponderada.
- Reutilizar transacoes, proventos, posicoes historicas e cotacoes da Sprint 12.
- Consolidar BUY, SELL e INCOME por data.
- Implementar composicao por periodo e agregacoes mensal/anual.
- Criar endpoint e validar propriedade da carteira.
- Criar testes unitarios, de servico e de controller.
- Executar toda a suite de testes do backend.

### Frontend

- Criar tipos TypeScript e servico de integracao.
- Adicionar cards de Rentabilidade total, Ultimos 12 meses e Ultimo mes.
- Adicionar grafico historico percentual.
- Adicionar tabela mensal/anual.
- Tratar carregamento, ausencia de dados, erros e responsividade.
- Executar npm run build.

## 9. QA funcional previsto

- Criar uma carteira de teste do zero com valores de facil conferencia manual.
- Validar compra inicial e valorizacao/desvalorizacao.
- Validar nova compra sem rentabilidade artificial.
- Validar venda sem perda artificial.
- Validar provento como retorno.
- Conferir total, ultimos 12 meses, ultimo mes, grafico e tabela mensal.

## 10. Fora do escopo da Sprint 13

- Benchmarks como CDI, IPCA, IBOV, IFIX, SMLL, IDIV e IVVB11.
- Custos operacionais, corretagem, emolumentos e outros custos.
- Controle de caixa ou saldo de corretora.
- DEPOSIT, WITHDRAWAL e fluxos de caixa independentes.
- Integracao com B3 ou corretoras.
- Rebalanceamento e metas de alocao.
- Evolucao do modelo especifico de Renda Fixa.
- Atualizacao geral de README.md e docs/PROJECT\_HISTORY.md.

## 11. Decisoes fechadas antes da implementacao

- Decisao 1: usar rentabilidade ponderada/TWR como metodologia da Sprint 13.
- Decisao 2: BUY e SELL serao neutralizados matematicamente, sem controle de saldo.
- Decisao 3: INCOME sera considerado retorno do investimento.
- Decisao 4: nao criar DEPOSIT/WITHDRAWAL ou modulo de caixa.
- Decisao 5: custos operacionais ficam para evolucao futura.
- Decisao 6: benchmarks ficam para uma Sprint posterior.

## **12. Definition of Done (DoD)**

- Formula validada por testes numericos.
- Backend compilando e suite completa de testes aprovada.
- Frontend compilando sem erros.
- API, cards, grafico e tabela validados funcionalmente.
- QA funcional concluido com conferencia manual dos cenarios principais.
- Alteracoes versionadas e enviadas aos repositorios.
- Working trees limpas e sincronizadas com origin/main.

## **13. Status inicial**

Status: Planejada - regras, escopo e decisoes pre-implementacao consolidadas.  
Implementacao ainda nao iniciada.